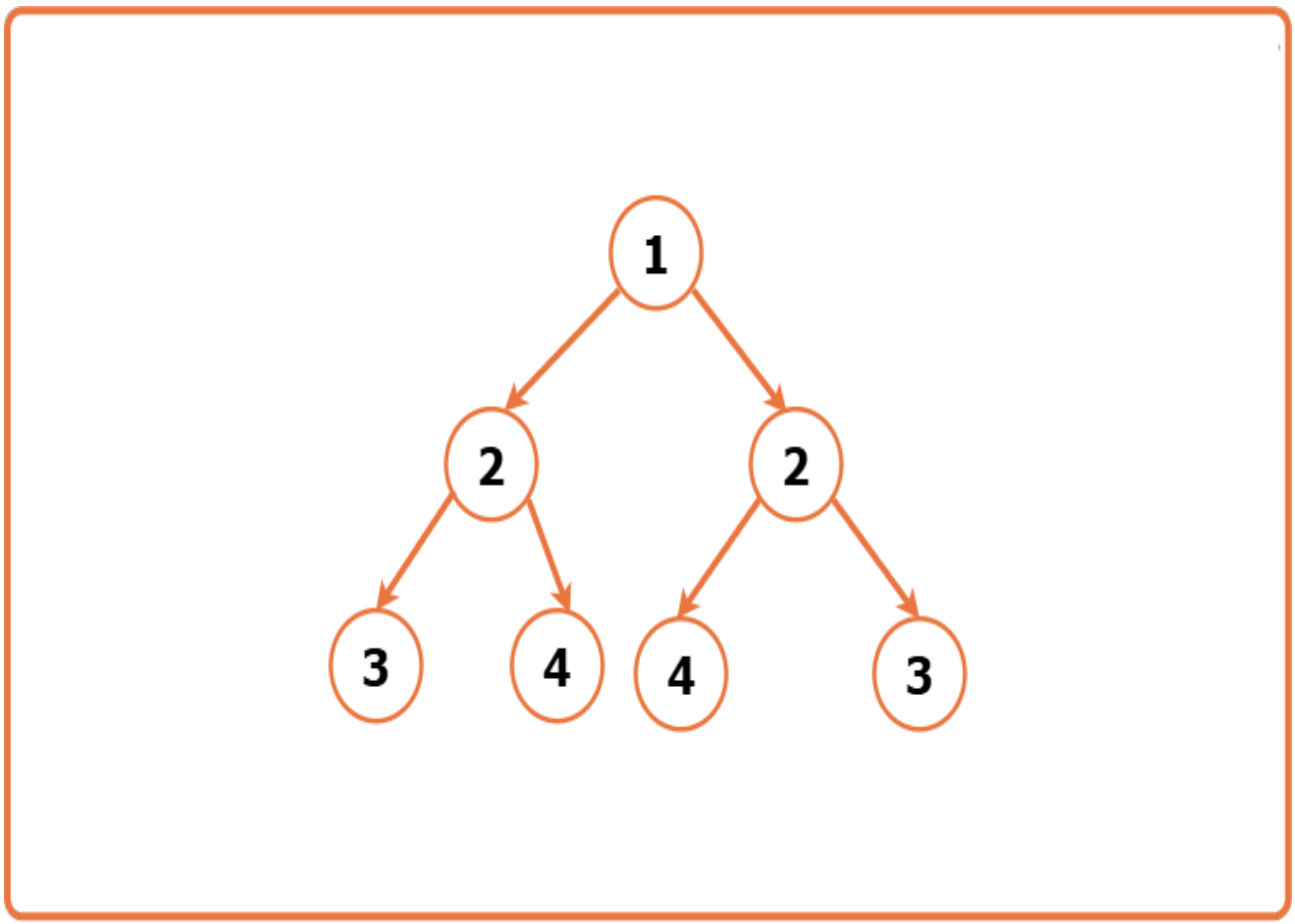
Check for Symmetrical Binary Tree

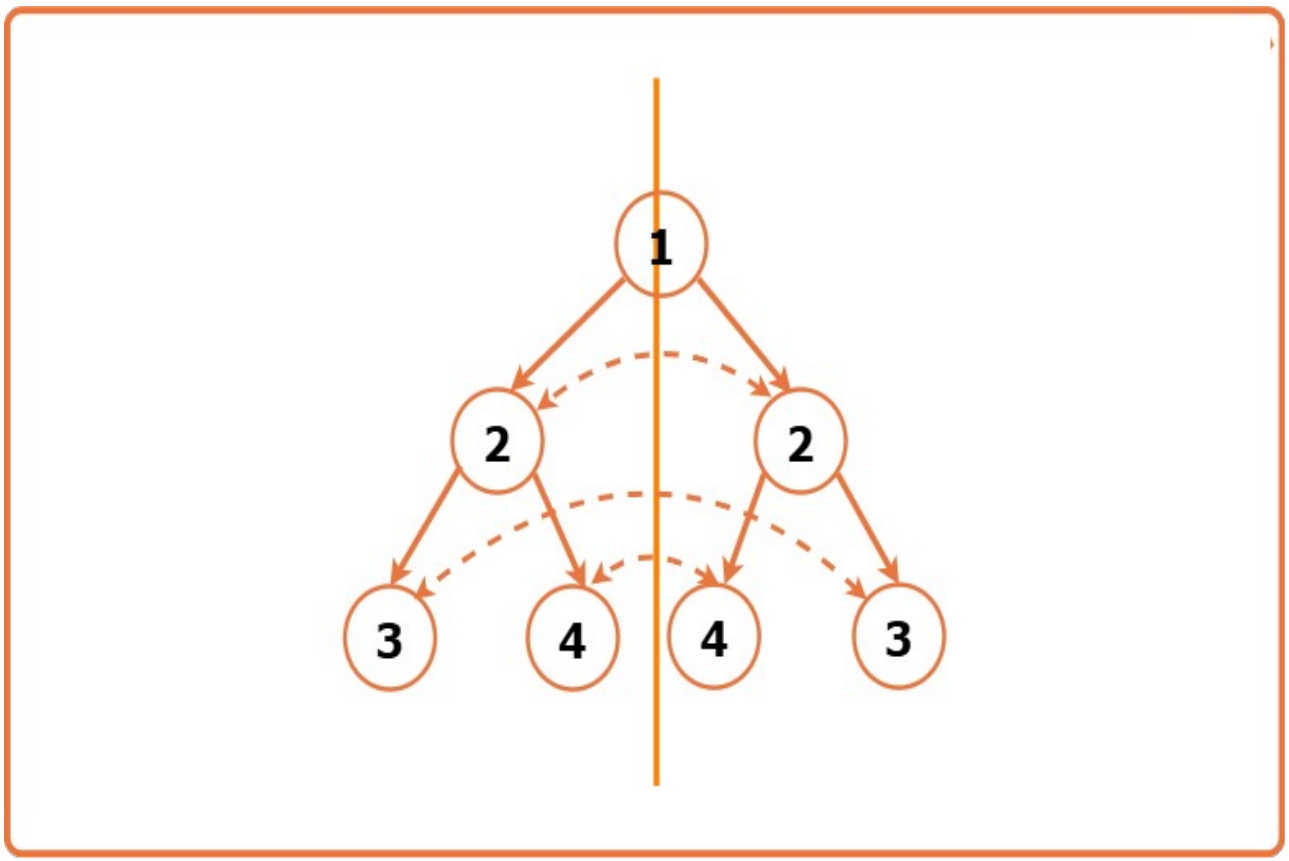
Problem Statement: Given a Binary Tree, determine whether the given tree is symmetric or not. A Binary Tree would be Symmetric, when its mirror image is exactly the same as the original tree. If we were to draw a vertical line through the centre of the tree, the nodes on the left and right side would be mirror images of each other.

Examples

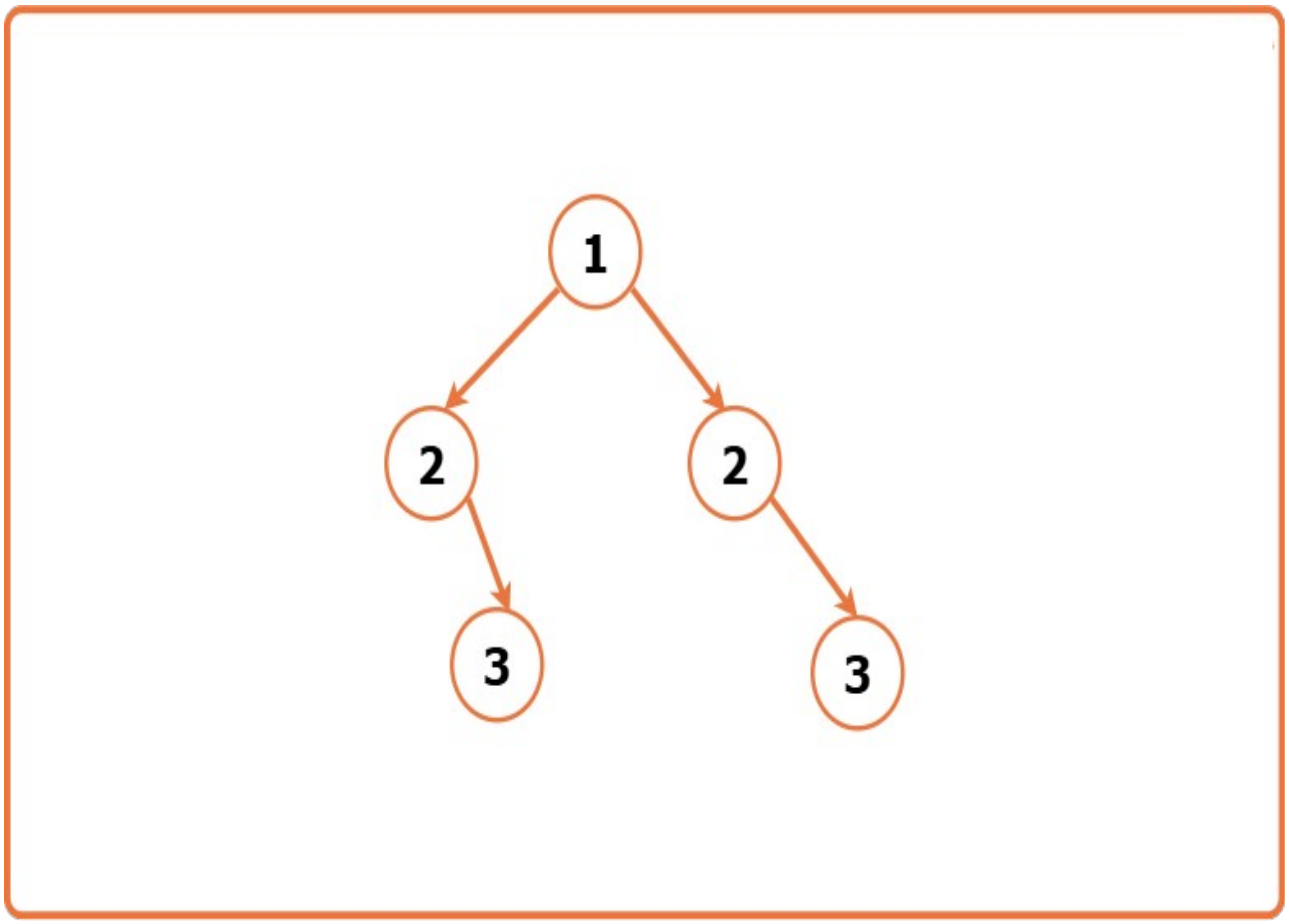
```
Example 1:
Input: Binary Tree: 1 2 2 3 4 4 3
```



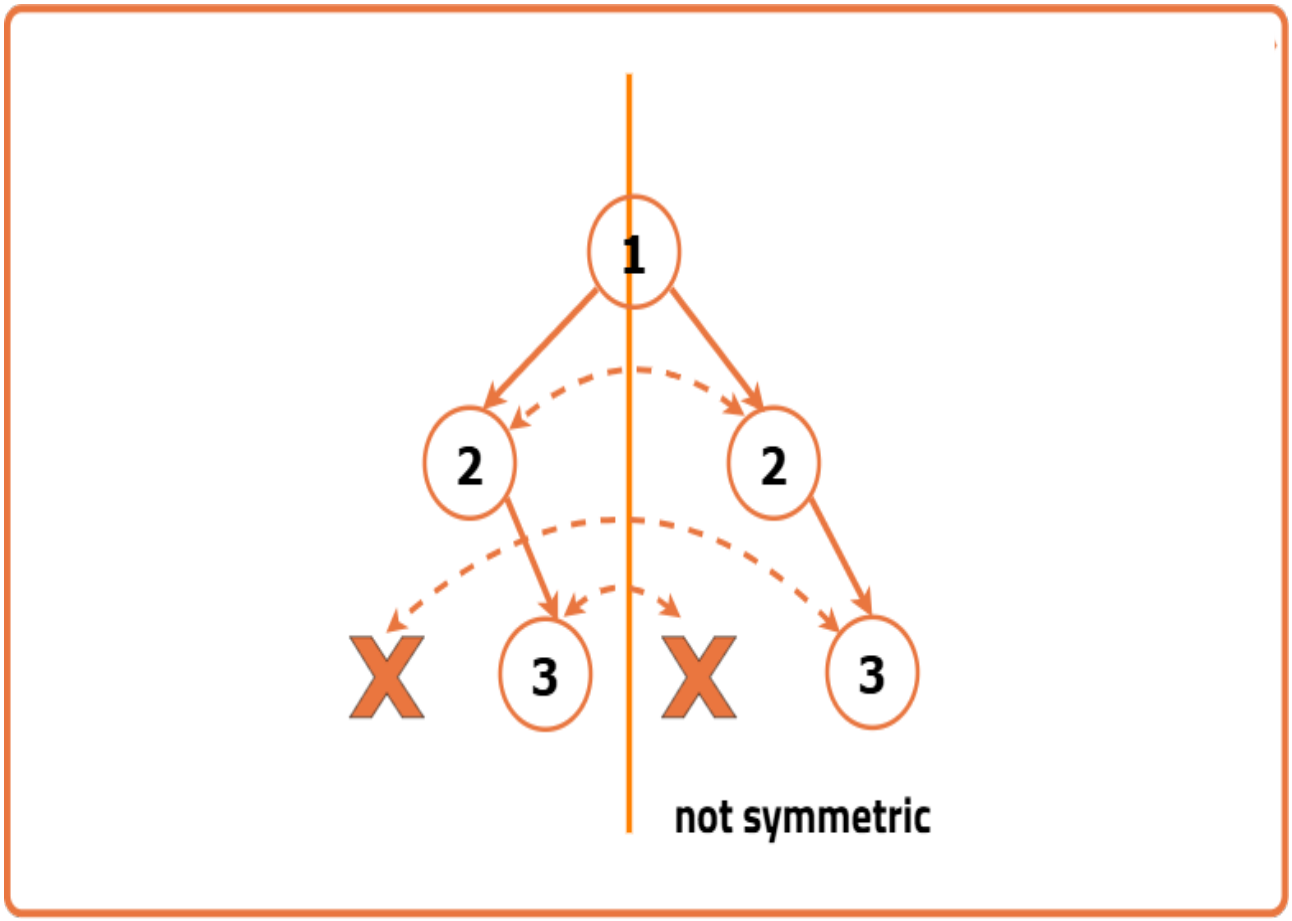

```
Output:True
Explanation: If we were to draw a vertical line through the centre of the tree,
dividing it into left and right parts, we observe that the nodes on the left and
right sides are mirror images of each other.
```



Example 2:
Input:Binary Tree: 1 2 2 -1 3 -1 3



```
Output : False
Explanation: While the tree has a symmetric structure at the first level with
nodes 2 and 2, the subtrees under nodes 2 and 2 are not symmetric. If we were to
draw a vertical line through the centre of the tree, dividing it into left and
right parts, we observe that the nodes on the left and right sides are not
mirror images of each other.
```



Approach
Algorithm

A binary tree is symmetric if its left and right sides are mirror images of each other. If a vertical line is drawn through the center, both sides should align perfectly.

Symmetry conditions:

1. The tree must visually mirror itself from left to right.
2. This mirror pattern must be consistent at every level of the tree.

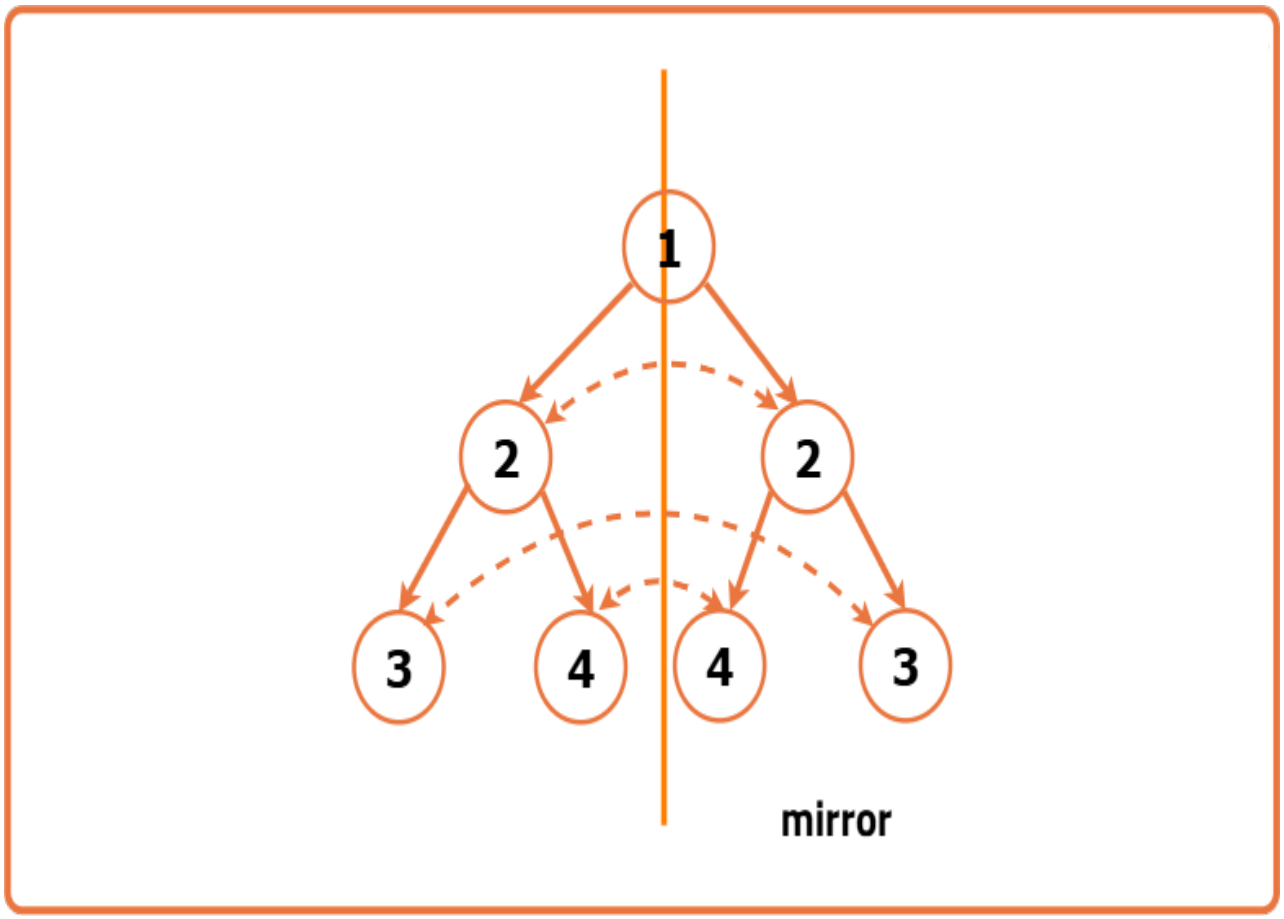
We compare left and right parts in a mirrored way - left child of the left side is compared with the right child of the right side, and vice versa.

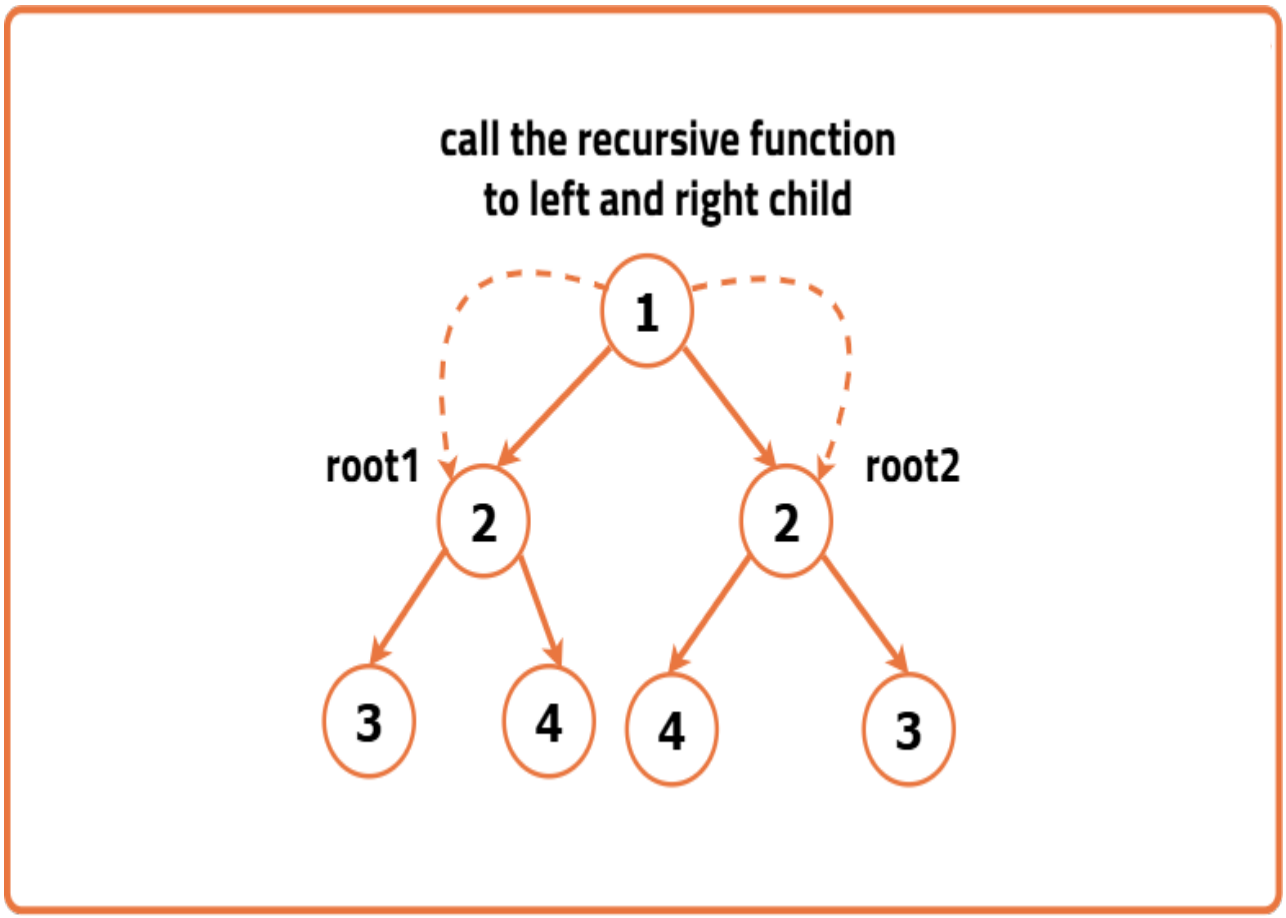
Base check: If both parts are empty, it is symmetric. If only one is empty, it's not.

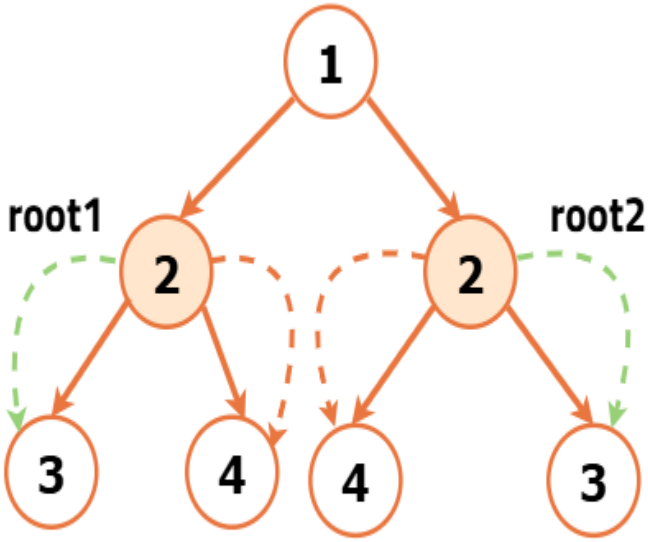
Mirror checks:

1. Both nodes should have the same value.
2. Left of left side matches right of right side.
3. Right of left side matches left of right side.

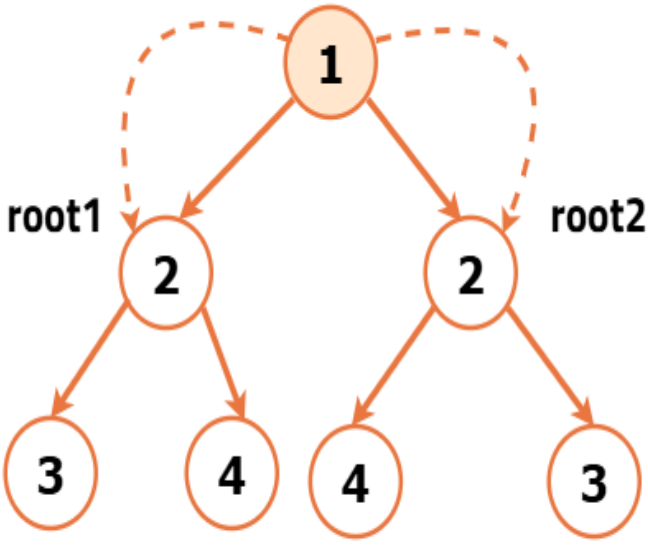
Final result: The tree is symmetric if all mirrored comparisons pass successfully from top to bottom.



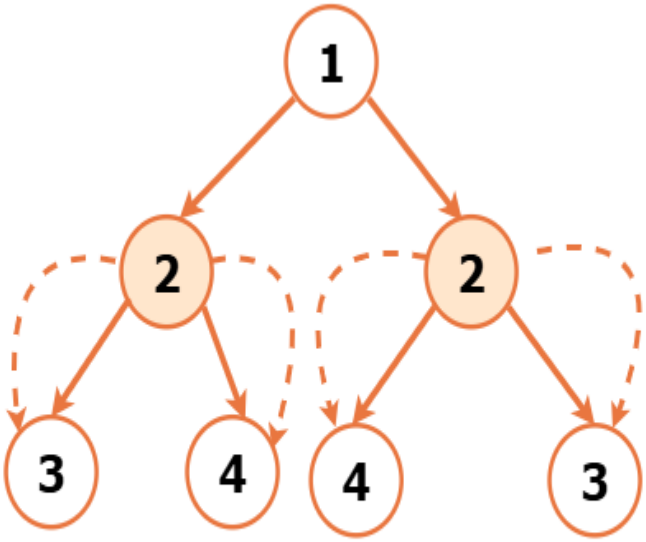




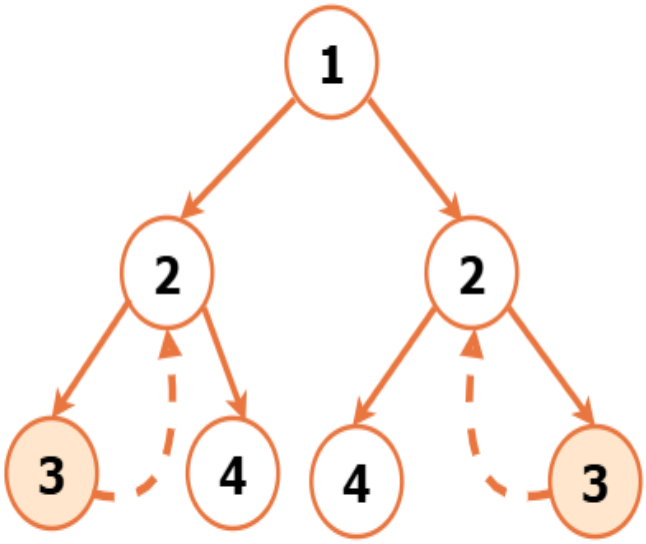
check value of nodes and then call the
function of left and right child and vice versa



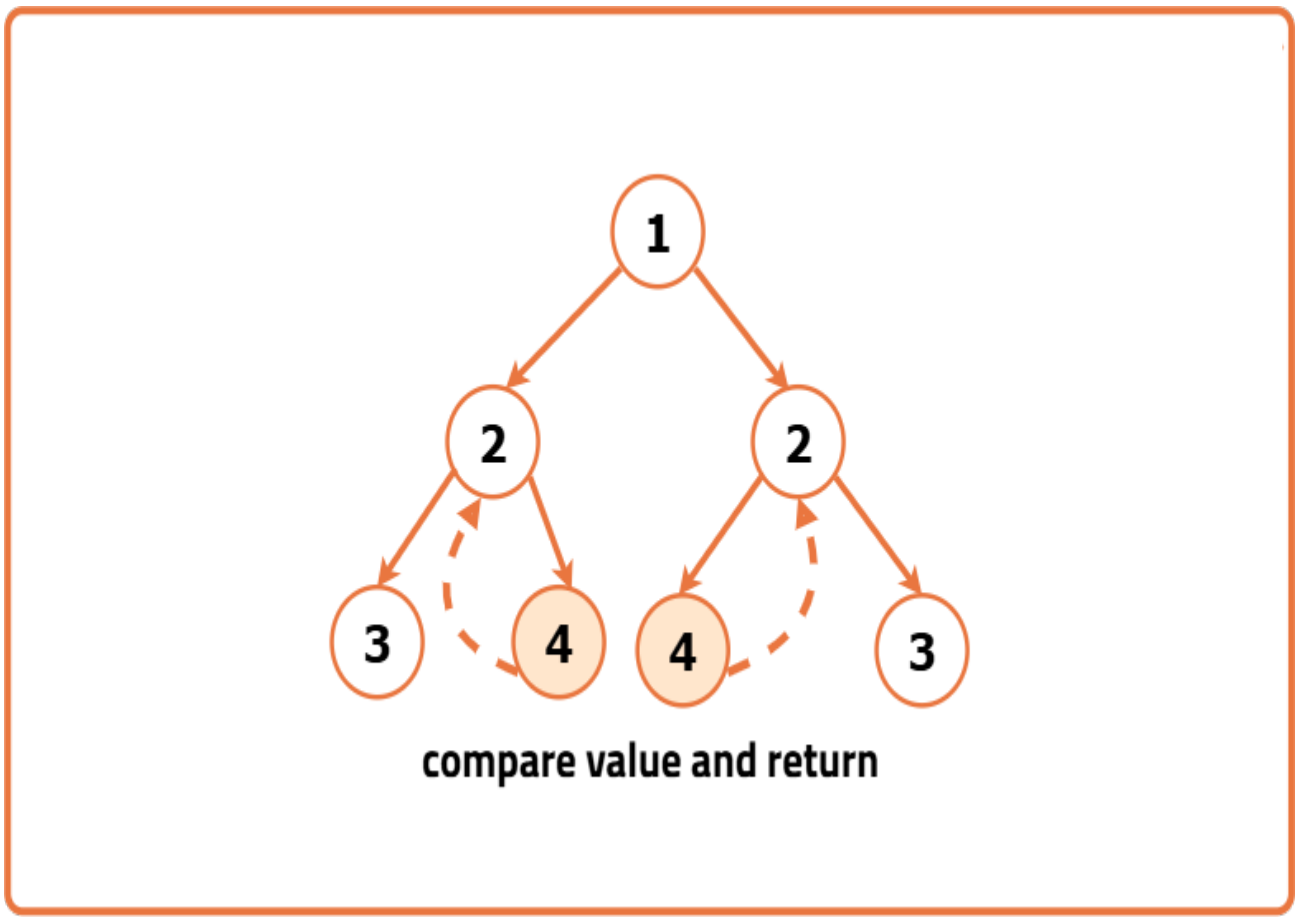
call symmetric function on
left and right child



compare value and check
left and right child



compare value and return



Code

```
// Node structure for the binary tree
class Node {
    int data;
    Node left;
    Node right;

    // Constructor to initialize
    // the node with a value
    public Node(int val) {
        data = val;
        left = null;
        right = null;
    }
}

class Solution {
    // Function to check if
    // two subtrees are symmetric
    private boolean isSymmetricUtil(Node root1, Node root2) {
        // Check if either subtree is null
        if (root1 == null || root2 == null) {
            // If one subtree is null, the other
            // must also be null for symmetry
            return root1 == root2;
        }
        // Check if the data in the current nodes is equal
        // and recursively check for symmetry in subtrees
        return (root1.data == root2.data)
                && isSymmetricUtil(root1.left, root2.right)
                && isSymmetricUtil(root1.right, root2.left);
    }
```

```
    // Public function to check if the
    // entire binary tree is symmetric
    public boolean isSymmetric(Node root) {
        // Check if the tree is empty
        if (root == null) {
            // An empty tree is
            // considered symmetric
            return true;
        }
        // Call the utility function
        // to check symmetry of subtrees
        return isSymmetricUtil(root.left, root.right);
    }
}

public class Main {
    // Function to print the Inorder
    // Traversal of the Binary Tree
    private static void printInorder(Node root) {
        if (root == null) {
            return;
        }
        printInorder(root.left);
        System.out.print(root.data + " ");
        printInorder(root.right);
    }

    public static void main(String[] args) {
        // Creating a sample binary tree
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(2);
        root.left.left = new Node(3);
        root.right.right = new Node(3);
        root.left.right = new Node(4);
        root.right.left = new Node(4);

        Solution solution = new Solution();

        System.out.print("Binary Tree (Inorder): ");
        printInorder(root);
        System.out.println();

        boolean res = solution.isSymmetric(root);

        if (res) {
            System.out.println("This Tree is Symmetrical");
        } else {
            System.out.println("This Tree is NOT Symmetrical");
        }
    }
}
```

Complexity Analysis

Time Complexity: O(N) where N is the number of nodes in the Binary Tree. This complexity arises from visiting each node exactly once during the traversal and the function compares the nodes in a symmetric manner.

Space Complexity: O(1) as no additional data structures or memory is allocated.